

移动应用开发

实验指导书（学生版）

Kotlin + Jetpack Compose 主线 | 微信小程序 / WebRTC 专题

课程属性	计算机科学与技术专业选修课
实验学时	16 学时
实验组成	实验 1-6 各 2 学时，实验 7 为 4 学时
版本	2026 学年教学使用稿

请按实验顺序完成并保留全过程 *Git* 记录

一、使用说明

本书用途 本学生版用于实验前预习、课堂操作、实验验收和课后提交。实验 1-6 各 2 学时，实验 7 为 4 学时；所有实验默认使用课程统一模板与 *Git* 仓库。

- 基础任务用于达成课程目标，必须完成；拓展任务为可选，用于展示更高水平。
- 每次实验至少保留 3 个有意义的 *Git* 提交节点：起始、主要功能、最终修复。禁止只在实验结束时一次性提交全部文件。
- 实验结束前准备回答“四问”：数据在哪里？状态在哪里？事件从哪里来、到哪里去？错误在哪里处理？
- 禁止把真实密码、*token*、*API Key* 或个人隐私数据提交到仓库、截图和实验报告。
- 可合理使用 *AI* 辅助检索、调试和样板代码，但必须能解释、修改、测试并对最终代码负责。

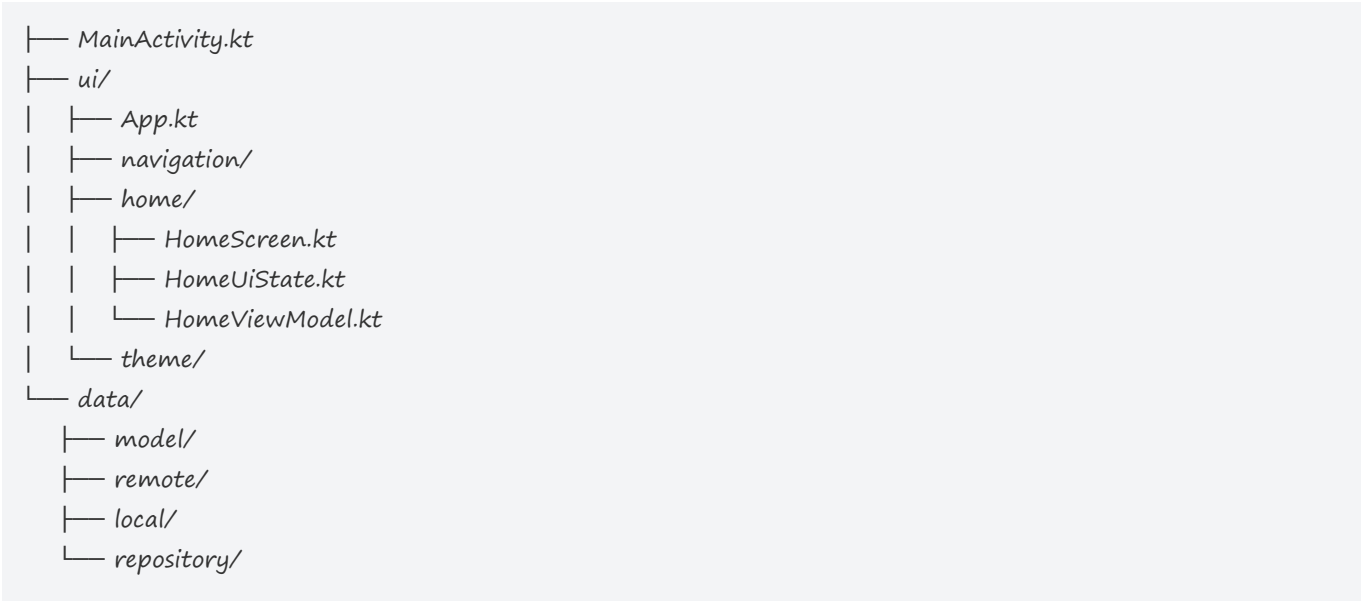
二、实验安排

实验	名称	学时	核心能力
1	<i>Kotlin</i> 与 <i>Compose</i> 开发环境及基础界面	2	环境、 <i>Kotlin</i> 、空安全、组件
2	移动 UI 设计与页面导航	2	<i>Compose</i> 布局、 <i>LazyColumn</i> 、 <i>Navigation</i>
3	<i>State</i> 、 <i>ViewModel</i> 与单向数据流	2	<i>UDF</i> 、 <i>UiState</i> 、 <i>StateFlow</i> 、 <i>Repository</i>
4	网络 <i>API</i> 、本地数据与移动端安全	2	<i>REST/JSON</i> 、 <i>DataStore</i> 、权限、安全
5	微信小程序开发	2	<i>WXML/WXSS</i> 、路由、网络封装
6	<i>WebRTC</i> 实时通信	2	媒体、 <i>PeerConnection</i> 、 <i>Offer/Answer/ICE</i> 、信令
7	移动应用综合项目	4	需求→设计→实现→测试→文档→答辩

三、统一工程与代码约定

3.1 Android 项目目录

edu.example.mobilecourse/



对象	统一规则	示例
Composable	PascalCase; 页面分 Route/Screen	HomeRoute / HomeScreen
UI 状态	XxxUiState, 不可变 data class	HomeUiState
ViewModel	XxxViewModel; 公开 StateFlow	HomeViewModel
Repository	接口 + 实现	TaskRepository / NetworkTaskRepository
导航目的地	XxxDestination	TaskDetailDestination
回调	on + 动作	onRefresh / onOpenTask

3.2 Route / Screen、UiState 与 Repository 模板

Route / Screen 统一模板

```
@Composable
fun XxxRoute(
    viewModel: XxxViewModel = viewModel(),
    onNavigate: (Int) -> Unit,
) {
    val uiState by viewModel.uiState.collectAsStateWithLifecycle()
```

```

XxxScreen(
    uiState = uiState,
    onAction = viewModel::onAction,
    onNavigate = onNavigate,
)
}

@Composable
fun XxxScreen(
    uiState: XxxUiState,
    onAction: (XxxAction) -> Unit,
    onNavigate: (Int) -> Unit,
) {
    // 纯 UI: 根据 uiState 渲染、把事件向上发送
}

```

UiState / Action 统一模板

```

data class XxxUiState(
    val isLoading: Boolean = false,
    val items: List<Item> = emptyList(),
    val errorMessage: String? = null,
)

sealed interface XxxAction {
    data object Refresh : XxxAction
    data class ItemClicked(val id: Int) : XxxAction
}

```

Repository 统一模板

```

interface ItemRepository {
    suspend fun getItems(): List<Item>
}

class FakeItemRepository : ItemRepository {
    override suspend fun getItems(): List<Item> = sampleItems
}

```

Loading / Error / Empty / Content 四态模板

```
when {  
    uiState.isLoading -> LoadingContent()  
    uiState.errorMessage != null -> ErrorContent(  
        message = uiState.errorMessage,  
        onRetry = onRetry,  
    )  
    uiState.items.isEmpty() -> EmptyContent()  
    else -> ItemList(uiState.items, onItemClick)  
}
```

3.3 Git 提交建议

类型	建议格式	示例
功能	<i>feat</i> : 描述功能	<i>feat: add task detail navigation</i>
修复	<i>fix</i> : 描述问题	<i>fix: handle empty task list</i>
重构	<i>refactor</i> : 描述结构调整	<i>refactor: extract TaskCard</i>
文档	<i>docs</i> : 描述文档	<i>docs: add lab 3 report</i>

实验 1 Kotlin 与 Compose 开发环境及基础界面（2 学时）

A. 实验目标

- 完成 *Android Studio*、*SDK*、模拟器/真机的基本环境验证。
- 创建 *Compose* 项目并能运行。
- 完成至少 3 种基本组件和一个可复用 *TaskCard*。
- 理解 *Kotlin data class* 与空安全在 UI 数据中的作用。

B. 实验准备

- 课程 *starter project* 或 *Android Studio Empty Activity (Compose)*。
- 准备 3 条 *CourseTask* 测试数据，其中 1 条 *owner=null*。
- 真机调试者提前开启开发者选项和 *USB* 调试。

C. 实验步骤

1. 运行 *Hello Compose*，截图记录设备/模拟器。
2. 创建 *CourseTask* 数据类和 *sampleTasks*。
3. 实现 *TaskCard(title, owner, completed, onClick)*。
4. 创建 2 个 *Preview*：进行中 / 已完成。
5. 在 *MainActivity* 中展示 3 个 *TaskCard*。
6. 把 *owner=null* 通过安全调用与 *Elvis* 运算符显示为“未分配”。

D. 起始/关键代码

```
data class CourseTask(  
    val id: Int,  
    val title: String,  
    val owner: String?,  
    val completed: Boolean = false,  
)  
  
fun displayOwner(task: CourseTask): String =  
    task.owner?.trim()?.takeIf { it.isNotEmpty() } ?: "未分配"
```

E. 验收点

- ☐ 项目可编译运行，无红色异常。
- ☐ *TaskCard* 能显示两种状态。
- ☐ 代码中无无理由 `!!`。
- ☐ 至少一个 *Composable* 可独立 *Preview*。

F. 提交物

- 源代码仓库链接或压缩包。
- 实验报告 1–2 页：环境、关键代码、运行截图、一个故障及解决方法。

G. 拓展任务（可选）

- 增加优先级样式。
- 使用 *MaterialTheme typography*，而不是大量硬编码字号。

H. 实验结束前自问

- 数据在哪里？
- 状态在哪里？
- 事件从哪里来、到哪里去？
- 错误在哪里处理？

提交提醒 实验报告只放关键代码与证据，不粘贴整个项目；至少记录一个真实故障的“现象 → 定位 → 原因 → 修复”。

实验 2 移动 UI 设计与页面导航（2 学时）

A. 实验目标

- 使用 *Column/Row/Card/LazyColumn/Modifier* 完成移动端列表。
- 完成 *Home* → *TaskDetail* 导航并传递 *taskId*。
- 理解“页面组件不持有导航控制器”的解耦方式。

B. 实验准备

- 实验 1 代码通过验收。
- 课程 *starter project* 已加入 *Navigation Compose*。

C. 实验步骤

7. 把首页改为 *LazyColumn*，使用稳定 *key*。
8. 新增 *EmptyContent*，当列表为空时显示。
9. 新增 *TaskDetailScreen(taskId)*。
10. 定义 *HomeDestination* 与 *TaskDetailDestination*。
11. 在 *App NavHost* 中连接两个目的地。
12. *TaskCard* 点击后调用 *onOpenTask(id)*，由上层 *navController.navigate*。
13. 测试返回键、无数据、100 条数据。

D. 起始/关键代码

```
LazyColumn(  
    verticalArrangement = Arrangement.spacedBy(8.dp),  
) {  
    items(tasks, key = { it.id }) { task ->  
        TaskCard(  
            title = task.title,  
            owner = task.owner,  
            completed = task.completed,  
            onClick = { onTaskClick(task.id) },  
        )  
    }  
}
```

E. 验收点

- □ 导航能带入正确 `taskId`。
- □ `TaskCard` 不直接访问 `NavController`。
- □ 列表为空时有明确空状态。
- □ 100 条数据滚动正常。

F. 提交物

- 运行录屏 30–60 秒或关键截图。
- 简图：`NavHost` / `Destination` / `NavController` 的关系。

G. 拓展任务（可选）

- 加入搜索框但先用本地过滤。
- 加入底部导航并保持页面职责清晰。

H. 实验结束前自问

- 数据在哪里？
- 状态在哪里？
- 事件从哪里来、到哪里去？
- 错误在哪里处理？

提交提醒 实验报告只放关键代码与证据，不粘贴整个项目；至少记录一个真实故障的“现象 → 定位 → 原因 → 修复”。

实验 3 State、ViewModel 与单向数据流（2 学时）

A. 实验目标

- 把屏幕状态从 *Composable* 移到 *ViewModel*。
- 使用 *HomeUiState* 表达 *Loading / Error / Empty / Content*。
- 使用 *StateFlow* 与 *collectAsStateWithLifecycle*。
- 解释 *State* 向下、*Event* 向上的方向。

B. 实验准备

- 实验 2 页面结构完成。
- 教师提供 *FakeTaskRepository*，可模拟 *delay* 和异常。

C. 实验步骤

14. 建立 *HomeUiState*。
15. 建立 *TaskRepository* 接口和 *FakeTaskRepository*。
16. 建立 *HomeViewModel*，初始化时调用 *refresh*。
17. 用 *viewModelScope.launch* 加载数据。
18. *HomeRoute* 负责 *collect StateFlow*；*HomeScreen* 变成纯 *UI*。
19. 给 *FakeRepository* 增加 *throw* 开关，测试 *Error + Retry*。
20. 把 *repository* 返回 *emptyList*，测试 *Empty*。

D. 起始/关键代码

```
data class HomeUiState(  
    val isLoading: Boolean = false,  
    val tasks: List<CourseTask> = emptyList(),  
    val errorMessage: String? = null,  
)  
  
// 统一方向：State 向下，Event 向上
```

E. 验收点

- ☐ *Screen* 不直接调用 *repository*。
- ☐ *ViewModel* 公开只读 *StateFlow*。

- □ 四种 *UI* 状态均可演示。
- □ 失败后可重试且不会崩溃。

F. 提交物

- 一张 *UDF* 数据流图。
- 关键代码: *UiState*、*ViewModel*、*Route/Screen*。
- 实验反思: 状态放错位置会发生什么。

G. 拓展任务（可选）

- 把 *Boolean* 组合改成 *sealed UI state*, 并比较优缺点。
- 为 *ViewModel* 写一个简单单元测试。

H. 实验结束前自问

- 数据在哪里？
- 状态在哪里？
- 事件从哪里来、到哪里去？
- 错误在哪里处理？

提交提醒 实验报告只放关键代码与证据，不粘贴整个项目；至少记录一个真实故障的“现象 → 定位 → 原因 → 修复”。

实验 4 网络 API、本地数据与移动端安全（2 学时）

A. 实验目标

- 通过 *Repository* 访问 REST API 并解析 JSON。
- 将网络结果映射成 *UiState*，处理错误与重试。
- 用 *DataStore* 保存一个用户偏好。
- 完成至少一个运行时权限或安全设计练习。

B. 实验准备

- 教师 *starter project* 预配置 *Retrofit* 与 *DataStore*。
- 默认示例使用无密钥教学 API；若校园网络不稳定，提供本地 *mock JSON*。

C. 实验步骤

21. 定义 *TodoDto* 与 *TodoApi*。
22. 实现 *NetworkTaskRepository*，把 DTO 转成 *CourseTask*。
23. 将 *HomeViewModel* 的 *repository* 替换为网络实现。
24. 模拟断网/错误地址，验证 *Error + Retry*。
25. 用 *DataStore* 保存“仅看未完成任务”或“深色模式”偏好。
26. 加入相机权限按钮：点击后再申请，不在启动时无理由申请。
27. 检查仓库中是否存在真实 API Key、密码、*token*。

D. 起始/关键代码

```
interface TodoApi {
    @GET("todos")
    suspend fun getTodos(): List<TodoDto>
}

data class TodoDto(
    val id: Int,
    val title: String,
    val completed: Boolean,
)
```

E. 验收点

- □ 网络成功和失败均有明确 UI。
- □ 数据访问仍通过 *Repository*。
- □ *DataStore* 不在 *Composable* 中直接读写。
- □ 权限拒绝后 *App* 不崩溃。
- □ 无敏感凭据提交到仓库。

F. 提交物

- 接口响应样例（删去敏感信息）。
- 成功/断网/拒绝权限三张截图。
- 安全检查清单。

G. 拓展任务（可选）

- 增加简单缓存策略。
- 用 *Room* 保存任务列表并支持离线浏览。

H. 实验结束前自问

- 数据在哪里？
- 状态在哪里？
- 事件从哪里来、到哪里去？
- 错误在哪里处理？

提交提醒 实验报告只放关键代码与证据，不粘贴整个项目；至少记录一个真实故障的“现象 → 定位 → 原因 → 修复”。

实验 5 微信小程序开发（2 学时）

A. 实验目标

- 掌握 WXML/WXSS/JS/JSON 页面结构。
- 完成列表与详情两页。
- 使用统一 `request` 封装访问网络。
- 比较小程序与 *Android* 的导航、状态和平台 API。

B. 实验准备

- 安装微信开发者工具。
- 导入课程 `mini-program-template`。
- 若真实网络域名校验影响实验，按教师提供的开发环境配置执行；上线规则单独说明。

C. 实验步骤

28. 阅读 `app.json` 并新增 `detail` 页面。

29. `index.wxml` 用 `wx:for` 显示列表。

30. `index.js` 调用 `request()`，维护 `isLoading/todos/error`。

31. 点击列表项使用 `wx.navigateTo` 传 `id`。

32. `detail` 页面读取 `options.id` 并显示。

33. 模拟错误地址，验证失败提示。

34. 填写 *Android* vs 小程序对照表。

D. 起始/关键代码

```
function request({ url, method = 'GET', data = {} }) {
  return new Promise((resolve, reject) => {
    wx.request({
      url: BASE_URL + url,
      method, data,
      success: (res) => {
        if (res.statusCode >= 200 && res.statusCode < 300) resolve(res.data)
        else reject(new Error(`HTTP ${res.statusCode}`))
      },
      fail: reject
    })
  })
}
```

E. 验收点

- □ 两页均可正常打开。
- □ 网络代码没有在多个页面复制 `wx.request`。
- □ 有 `loading/error` 状态。
- □ 能解释 `WXML` 与 `Compose` 在“数据驱动 UI”上的相似点。

F. 提交物

- 小程序源码。
- 运行截图。
- `Android` / 小程序比较表（至少 5 项）。

G. 拓展任务（可选）

- 增加下拉刷新。
- 增加 `tabBar` 并解释 `navigateTo` 与 `switchTab` 的区别。

H. 实验结束前自问

- 数据在哪里？
- 状态在哪里？
- 事件从哪里来、到哪里去？
- 错误在哪里处理？

提交提醒 实验报告只放关键代码与证据，不粘贴整个项目；至少记录一个真实故障的“现象 → 定位 → 原因 → 修复”。

实验 6 WebRTC 实时通信（2 学时）

A. 实验目标

- 获得摄像头/麦克风 *MediaStream*。
- 理解 *RTCPeerConnection* 的建立过程。
- 通过 *WebSocket* 观察 *Offer / Answer / ICE candidate*。
- 理解安全上下文、权限与 *STUN/TURN* 的作用边界。

B. 实验准备

- 安装 *Node.js*。
- 解压课程 *webrtc-template*，执行 *npm install*。
- 优先在同机 *localhost* 两个标签页完成基础验收。

C. 实验步骤

35. *npm start*，打开 *http://localhost:8080*。

36. 点击“获取媒体”，确认权限和 *localVideo*。

37. 打开第二标签页，也获取媒体。

38. 在一页发起呼叫，观察双方日志。

39. 在 *main.js* 中给 *onicecandidate / ontrack* 增加自定义日志。

40. 挂断并确认 *RTCPeerConnection* 关闭。

41. 讨论跨设备为何需要 *HTTPS/WSS*、*STUN/TURN*。

D. 起始/关键代码

```
const stream = await navigator.mediaDevices.getUserMedia({
  video: true,
  audio: true
})

const pc = new RTCPeerConnection()
stream.getTracks().forEach(track => pc.addTrack(track, stream))

pc.ontrack = event => {
  remoteVideo.srcObject = event.streams[0]
}
```

E. 验收点

- ☐ 本地视频可显示。
- ☐ 能在日志中区分 *offer*、*answer*、*candidate*。
- ☐ 能说明 *signaling* 负责“协商消息”而不是等同于媒体传输。
- ☐ 权限拒绝时有错误提示。

F. 提交物

- 关键日志截图。
- 手绘或流程图：*A/B/Signaling* 三者交互。
- 100–200 字说明 *STUN* 与 *TURN* 的不同用途。

G. 拓展任务（可选）

- 增加房间号。
- 增加 *mute/camera toggle*。
- 在教师提供的 *HTTPS/WSS* 环境完成跨设备通信。

H. 实验结束前自问

- 数据在哪里？
- 状态在哪里？
- 事件从哪里来、到哪里去？
- 错误在哪里处理？

提交提醒 实验报告只放关键代码与证据，不粘贴整个项目；至少记录一个真实故障的“现象 → 定位 → 原因 → 修复”。

实验 7 移动应用综合项目（4 学时）

A. 实验目标

- 以 2-4 人团队完成需求、设计、实现、测试、文档和答辩。
- 综合使用 *Compose*、导航、状态、网络/本地数据或设备能力。
- 体现版本管理、错误处理、安全隐私和团队协作。

B. 实验准备

- 提交 1 页项目 *Proposal*：用户、场景、3-5 个核心功能、技术点、分工。
- 建立 *Git* 仓库和 *Issues*/任务清单。
- 教师确认项目规模适合 4 学时集中实践 + 课外完善。

C. 实验步骤

42.Checkpoint 1: 5 分钟需求确认——明确核心用户路径，砍掉非核心功能。

43.Checkpoint 2: 20 分钟架构和数据流——画 *Screen / ViewModel / Repository*。

44.Checkpoint 3: 60 分钟核心功能结对实现——至少打通一条完整用户路径。

45.Checkpoint 4: 30 分钟错误与边界测试——空数据、无网络、权限拒绝、输入错误。

46.Checkpoint 5: 30 分钟 *UI/UX* 和可访问性检查。

47.Checkpoint 6: 20 分钟代码清理、*README*、演示脚本。

48.Checkpoint 7: 现场演示与问答。

D. 起始/关键代码

建议最低工程边界：

UI / Presentation: *Compose Screen*、*ViewModel*

Data: *Repository*、*Network / DataStore / Room*

Domain（可选）：复杂过滤、排序、统计等可复用规则

E. 验收点

- ☐ 具有可运行的核心用户路径。
- ☐ 至少使用导航 + 状态管理 + 网络/本地数据之一。
- ☐ 有 *Loading/Empty/Error* 等边界状态。
- ☐ 仓库提交能看出团队过程。

- 答辩成员能解释自己负责的核心代码。

F. 提交物

- 源代码仓库。
- README：项目简介、运行方式、架构、成员分工。
- 3-5 分钟演示。
- 项目文档：需求、界面、数据/架构、测试、安全说明。

G. 拓展任务（可选）

- 接入 AI、地图、视觉、物联网、WebRTC 等创新能力，但须能解释原理和数据安全。

H. 实验结束前自问

- 数据在哪里？
- 状态在哪里？
- 事件从哪里来、到哪里去？
- 错误在哪里处理？

提交提醒 实验报告只放关键代码与证据，不粘贴整个项目；至少记录一个真实故障的“现象 → 定位 → 原因 → 修复”。

四、评分 Rubric（学生公开版）

评分原则 Rubric 在实验开始前即公开。成绩不只看“能否跑起来”，还看原理解、代码质量、实验记录和独立调试规范。

维度	权重	优秀（90-100）	良好（80-89）	达标（60-79）	未达标（<60）
功能与验收点	40	全部基础任务完成；边界情况可演示	主要功能完成；少量非核心问题	能完成核心路径；存在明显缺项	核心路径不可运行或大量缺失
原理解	20	能解释状态/数据/事件/错误流，并回答追问	能解释主要原理	能描述做法但解释较浅	无法解释关键代码
代码质量	15	结构清晰、命名统一、无明显重复和危险写法	结构基本清晰	可运行但耦合/重复较多	代码混乱或大量复制粘贴

维度	权重	优秀 (90-100)	良好 (80-89)	达标 (60-79)	未达标 (<60)
实验记录/报告	15	截图、关键代码、问题定位、反思完整	记录较完整	有基本截图与说明	缺少关键证据或报告
独立调试与规范	10	能定位问题; <i>Git</i> /安全/提交规范完整	大部分能独立处理	较依赖教师但能完成	无法独立完成或存在不规范行为

4.1 综合项目 Rubric (100 分)

指标	权重	优秀	良好	达标	未达标
功能完整性与稳定性	25	核心流程完整; 边界/异常有处理; 演示稳定	主要功能完整; 少量非阻断问题	可完成基本流程但缺边界处理	核心流程无法稳定运行
技术实现与应用架构	20	<i>UI/Data</i> 职责清晰; <i>UDF/ViewModel/Repository</i> 使用合理	架构基本合理; 少量耦合	有基本分层但职责混杂	业务逻辑大量堆在 <i>UI</i> , 结构失控
<i>UI</i> 设计与用户体验	15	层级清楚、交互一致、可读/可触达、错误反馈明确	整体合理; 个别细节不一致	能用但界面和反馈较粗糙	严重影响使用或缺关键反馈
网络、数据或设备能力	15	真实解决业务需求; 错误/权限/离线策略合理	功能正确; 边界处理基本具备	仅完成基础调用	调用不稳定或无法解释数据流
代码质量、版本管理、测试与安全	10	命名/复用/ <i>Git</i> /测试/安全检查完整	大部分规范到位	代码可读但规范与测试较弱	大量复制、敏感信息泄露或无版本过程
项目创新性	5	创新服务于用户问题且实现可解释	有明确特色	主要为常规功能组合	无明显创新或堆砌技术
项目文档	5	<i>README</i> /架构/测试/运行说明完整可复现	文档较完整	有基本说明	缺失或无法按文档运行
项目答辩与团队协作	5	分工证据清楚; 成员均能解释核心实现	分工较清楚; 回答大部分问题	部分成员贡献/理解较弱	无法说明分工或存在明显代做风险

AI 辅助开发 允许合理使用 **AI** 进行检索、调试、样板代码和文档辅助; 评分看学生是否能解释、修改、测试并对最终代码负责。若核心代码无法解释, 即使功能可运行, 也不能在“原理理解/架构/答辩”维度获得高分。

五、实验报告统一模板

栏目	要求
1. 实验目标	用自己的话写 2-4 条，不复制整段指导书
2. 环境	<i>Android Studio</i> /设备/工具版本；必要时记录网络环境
3. 设计	页面/数据流/类关系简图；说明关键决策
4. 实现	只放关键代码，不粘贴整个项目
5. 结果	成功截图 + 至少一个边界/错误状态
6. 故障与调试	记录一个真实问题：现象→定位→原因→修复
7. 思考	回答该实验的原理问题或比较题

六、提交前自检清单

- ☐ 项目可从干净环境按 *README* 启动。
- ☐ 没有真实密码、*token*、*API Key*、个人隐私数据进入仓库。
- ☐ 核心页面覆盖 *Loading / Empty / Error / Content* 中适用的状态。
- ☐ *Compose Screen* 不直接访问网络或 *DataStore*。
- ☐ *ViewModel* 公开只读状态；事件命名清晰。
- ☐ 导航参数正确，返回行为正常。
- ☐ 权限只在需要时申请，拒绝后有可理解的反馈。
- ☐ 至少测试一个失败场景。
- ☐ *Git* 历史能体现开发过程。
- ☐ 答辩成员能解释本人核心代码。